

Design and Implementation of a Fault-Tolerant Distributed File Storage System

Feasibility Study, Requirements Specification, and Database Design

Course: Distributed Systems

Project type: Semester-end mini project

Prepared by: _____

Team members: _____

Institution: _____

Supervisor: _____

Date: _____

Abstract

This project proposes the design and implementation of a prototype distributed file storage system. The system divides an uploaded file into smaller chunks and distributes those chunks across multiple storage nodes. A metadata server maintains the relationship between logical files, ordered chunks, cryptographic hashes, and physical node locations. During download, the system retrieves, verifies, and reassembles the chunks to reproduce the original file.

The proposed system addresses three important distributed-storage problems: storage distribution, data integrity, and node failure. SHA-256 hashing detects accidental corruption, chunk-level deduplication prevents identical chunks from being stored repeatedly, and replication factor two allows the system to continue serving data after the loss of one storage node. The project is designed as an educational prototype using Python, FastAPI, SQLite, and independent storage-node processes. Optional erasure coding is proposed as an advanced extension because it can provide fault tolerance with less storage overhead than full replication.

Keywords: distributed systems, distributed file storage, chunking, deduplication, replication, fault tolerance, SHA-256, metadata server.

1. Introduction

A conventional file-storage application normally stores a complete file on one server. This approach is simple, but it creates a single point of failure and may become inefficient when files are large or when many users upload data. A distributed file-storage system divides data across multiple storage nodes and maintains metadata that allows the original file to be located and reconstructed.

The design follows the general idea used in distributed file systems such as HDFS. HDFS separates filesystem metadata from application data: a master service manages namespace and

block information, while DataNodes store the actual blocks. [1]

This project does not attempt to reproduce a production system such as HDFS, Ceph, or a cloud-storage platform. Instead, it implements a small, understandable prototype that demonstrates the essential principles of distributed storage.

2. Problem statement

Traditional single-server storage has the following limitations:

- Failure of the storage server can make a file unavailable.
- A single disk may not have sufficient capacity.
- Large files are difficult to transfer and manage as one object.
- Repeated files or repeated file sections waste storage space.
- Data corruption may remain undetected without integrity verification.
- A system without metadata cannot efficiently locate distributed file parts.

Therefore, this project addresses the following problem:

How can a file be divided, distributed, identified, protected, and reconstructed across multiple storage nodes while reducing unnecessary storage consumption?

3. Aim and objectives

3.1 Aim

To design and implement a prototype distributed file-storage system that provides chunk-based storage, file reconstruction, integrity verification, deduplication, and basic fault tolerance.

3.2 Objectives

1. Design a client-server architecture for distributed file storage.
2. Divide files into fixed-size chunks.
3. Generate SHA-256 hashes for files and chunks.
4. Distribute chunks across multiple storage nodes.
5. Store metadata about file order and chunk locations.
6. Reconstruct the original file during download.
7. Detect corrupted chunks through hash comparison.
8. Reduce storage consumption through chunk-level deduplication.
9. Improve availability by maintaining a second copy of each chunk.
10. Detect failed nodes through heartbeat monitoring.
11. Recover affected chunks on healthy nodes.

12. Evaluate upload time, download time, storage overhead, and recovery time.

4. Proposed system

4.1 High-level operation

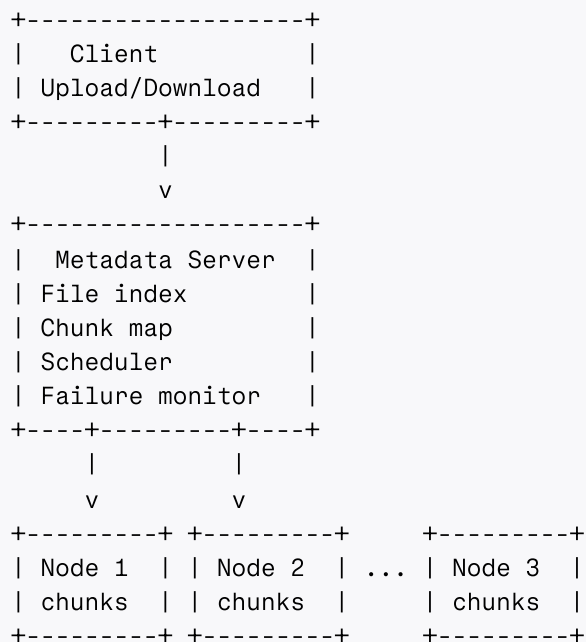
Upload:

File -> Chunking -> Hashing -> Deduplication -> Placement -> Replication -> Metadata

Download:

Metadata -> Locate chunks -> Retrieve -> Verify -> Order -> Reconstruct file

4.2 System architecture



4.3 Component responsibilities

Component	Responsibility
Client	Selects files and requests upload/download operations
Metadata server	Maintains file, chunk, user, node, and location information
Scheduler	Selects healthy nodes for chunk placement
Storage node	Stores and returns binary chunks
Deduplication service	Checks whether a chunk already exists
Replication service	Stores a backup copy on another node
Monitoring service	Sends heartbeat requests and detects node failure

Component	Responsibility
Recovery service	Creates replacement copies after failure

5. Functional requirements

FR-01: User registration and authentication

The system shall allow a user to register and log in. Passwords shall be stored as hashes rather than plain text.

FR-02: File upload

The system shall allow an authenticated user to upload a file.

FR-03: File chunking

The system shall divide each file into fixed-size chunks. The initial configuration shall use a chunk size of 1 MB.

FR-04: Chunk hashing

The system shall calculate a SHA-256 hash for each chunk and for the complete file.

FR-05: Chunk placement

The system shall distribute chunks among healthy storage nodes.

FR-06: Deduplication

The system shall avoid storing a chunk again when an identical chunk hash already exists.

FR-07: Replication

The system shall maintain two physical copies of each unique chunk on different nodes.

FR-08: File download

The system shall locate chunks, verify their hashes, arrange them in sequence, and reconstruct the original file.

FR-09: Failure detection

The system shall monitor storage-node availability using heartbeat requests.

FR-10: Failure recovery

The system shall create a replacement copy on a healthy node when a replicated chunk loses one of its copies.

FR-11: File deletion

The system shall remove the file's metadata references and decrement chunk reference counts.

FR-12: Monitoring

The system shall display node status, capacity, used storage, and number of stored chunks.

6. Non-functional requirements

Requirement	Target for prototype
Correctness	Reconstructed file must be byte-for-byte identical
Integrity	Every retrieved chunk must pass SHA-256 verification
Availability	A file remains downloadable after one node fails if a replica exists
Performance	Upload and download times shall be measured experimentally
Scalability	Design should permit additional storage nodes
Usability	Basic GUI or web interface with clear status messages
Maintainability	Separate client, metadata, node, and common modules
Security	Authentication, authorization, input validation, and safe file paths
Recoverability	Lost replicas shall be recreated after node recovery detection

7. Feasibility study

7.1 Technical feasibility

The project is technically feasible for a semester prototype. Python provides file I/O, hashing, HTTP communication, concurrency, and database libraries. FastAPI can expose REST endpoints, and SQLite can store metadata without requiring a separate database server.

The prototype can run three storage nodes on one computer using different ports and separate directories. For a more realistic demonstration, each node can run on a different computer or virtual machine.

Technical item	Proposed implementation	Assessment
Client	Tkinter, command line, or web interface	Feasible
Metadata server	Python and FastAPI	Feasible

Technical item	Proposed implementation	Assessment
Storage nodes	Independent FastAPI processes	Feasible
Metadata database	SQLite	Feasible
Hashing	Python <code>hashlib</code> SHA-256	Feasible
Communication	HTTP/REST using <code>requests</code> or <code>httpx</code>	Feasible
Replication	Two copies on separate nodes	Feasible
Node monitoring	Periodic health requests	Feasible
Erasure coding	Reed-Solomon library	Possible but advanced

7.2 Economic feasibility

The system can be developed using free and open-source software. Existing laptops can simulate the cluster. No cloud service is necessary for the demonstration.

Cost item	Requirement
Software	Free and open-source tools
Hardware	Existing laptop or desktop
Network	Local network or localhost ports
Database	SQLite, no license cost
Hosting	Not required for prototype
Main cost	Team development time

A production system would require additional disks, network equipment, secure deployment, monitoring, backups, and maintenance. Those costs are outside the semester scope.

7.3 Operational feasibility

The proposed operations are simple for a user:

1. Log in.
2. Select a file.
3. Click Upload.
4. View the file list.
5. Select Download.

Distribution, deduplication, hash checking, and replication occur automatically in the background. An administrator can view node health and recovery information.

7.4 Schedule feasibility

The MVP can be completed in eight weeks if the team develops in stages. Erasure coding, encryption, and advanced content-defined chunking should be implemented only after the core system passes upload and download tests.

7.5 Legal and ethical feasibility

Testing should use team-owned or permission-approved files. The system should not be used to distribute copyrighted, confidential, or malicious data. Authentication and ownership controls should prevent unauthorized downloads.

7.6 Storage feasibility

With replication factor two, storage use is approximately twice the logical data size. Deduplication can significantly reduce this value when files share identical chunks.

An optional (4,2) erasure-coded layout uses four data chunks and two parity chunks. Its theoretical storage factor is:

$$(4 + 2) / 4 = 1.5$$

Thus, 100 MB of logical data requires approximately 150 MB before additional metadata and implementation overhead. Erasure coding is space-efficient, but it requires more complex encoding, decoding, and recovery logic. Ceph documentation describes the same data-chunk and coding-chunk principle. [2]

7.7 Feasibility conclusion

The project is feasible technically, economically, operationally, legally, and within the semester schedule. The recommended implementation is fixed-size chunking, SHA-256 verification, chunk deduplication, replication factor two, and failure recovery. Erasure coding is an optional extension rather than a dependency for project completion.

8. Data model and ER diagram

The database stores metadata, not the complete binary data. Actual chunk bytes are stored in node-specific directories.

```
erDiagram
    USER ||--o{ FILE : owns
    FILE ||--|{ FILE_CHUNK : contains
    CHUNK ||--o{ FILE_CHUNK : referenced_by
    CHUNK ||--o{ CHUNK_LOCATION : stored_at
    STORAGE_NODE ||--o{ CHUNK_LOCATION : contains
    STORAGE_NODE ||--o{ HEARTBEAT : reports
    USER ||--o{ DOWNLOAD_LOG : performs
    FILE ||--o{ DOWNLOAD_LOG : records
```

```
USER {
    string user_id PK
    string full_name
    string email UK
    string password_hash
    datetime created_at
}
FILE {
    string file_id PK
    string user_id FK
    string file_name
    bigint file_size
    string file_hash UK
    int chunk_size
    int total_chunks
    string status
    datetime created_at
}
CHUNK {
    string chunk_hash PK
    int chunk_size
    int reference_count
    datetime created_at
}
FILE_CHUNK {
    string file_id PK, FK
    int sequence_number PK
    string chunk_hash FK
}
STORAGE_NODE {
    string node_id PK
    string host
    int port
    bigint capacity_bytes
    bigint used_bytes
    string status
    datetime last_seen
}
CHUNK_LOCATION {
    string chunk_hash PK, FK
    string node_id PK, FK
    string copy_type
    string status
    datetime stored_at
}
HEARTBEAT {
    int heartbeat_id PK
    string node_id FK
    string status
    bigint free_bytes
    datetime checked_at
}
DOWNLOAD_LOG {
    int download_id PK
    string user_id FK
    string file_id FK
}
```

```
    string status
    int duration_ms
    datetime downloaded_at
}
```

Relationship interpretation

- One user can own many files.
- One file contains many ordered file-chunk records.
- One unique chunk can be referenced by many files, enabling deduplication.
- One chunk can be stored on multiple nodes, enabling replication.
- One node produces many heartbeat records.
- A user can create many download records.

Relational schema

```
USER(user_id PK, full_name, email UNIQUE, password_hash, created_at)
FILE(file_id PK, user_id FK, file_name, file_size, file_hash UNIQUE,
     chunk_size, total_chunks, status, created_at)
CHUNK(chunk_hash PK, chunk_size, reference_count, created_at)
FILE_CHUNK(file_id PK/FK, sequence_number PK, chunk_hash FK)
STORAGE_NODE(node_id PK, host, port, capacity_bytes, used_bytes,
             status, last_seen)
CHUNK_LOCATION(chunk_hash PK/FK, node_id PK/FK, copy_type, status, stored_at)
HEARTBEAT(heartbeat_id PK, node_id FK, status, free_bytes, checked_at)
DOWNLOAD_LOG(download_id PK, user_id FK, file_id FK, status,
             duration_ms, downloaded_at)
```

9. Algorithms

9.1 Upload algorithm

```
Input: file F
1. Calculate file_hash.
2. Create file_id.
3. Read F in fixed-size chunks.
4. For each chunk at sequence i:
    a. Calculate chunk_hash.
    b. Query metadata for chunk_hash.
    c. If it exists, increase reference count.
    d. Otherwise select two healthy nodes and store the chunk.
    e. Record chunk locations.
    f. Record file_id, sequence i, and chunk_hash.
5. Mark the file COMPLETE after all chunks are verified.
6. Return file_id to the client.
```

9.2 Download algorithm

Input: `file_id`

1. Retrieve file metadata.
2. Retrieve ordered chunk records.
3. For each chunk:
 - a. Request the primary copy.
 - b. If unavailable, request the replica.
 - c. Calculate the received hash.
 - d. Reject the chunk if the hash is incorrect.
4. Join chunks in sequence order.
5. Calculate the reconstructed file hash.
6. Compare it with the original `file_hash`.
7. Return the file only if verification succeeds.

9.3 Recovery algorithm

1. Mark an unresponsive node as FAILED.
2. Find chunks stored on the failed node.
3. Find a valid surviving copy for each chunk.
4. Select a healthy replacement node.
5. Copy the chunk to the replacement node.
6. Verify its SHA-256 hash.
7. Update `CHUNK_LOCATION`.
8. Mark the recovery operation successful.

10. Technology stack

Layer	Technology
Programming language	Python
API framework	FastAPI
Server	Uvicorn
Database	SQLite
Client	Tkinter, command-line, or HTML interface
Hashing	Python <code>hashlib</code>
HTTP client	<code>requests</code> or <code>httplib</code>
Testing	<code>pytest</code> and scripted failure tests
Version control	Git and GitHub
Optional parity	Reed-Solomon library

11. Testing and evaluation

Functional tests

- Upload a file smaller than one chunk.
- Upload a file larger than one chunk.
- Download and compare the result byte by byte.
- Upload the same file twice and verify reuse.
- Upload files with common content and measure deduplication.
- Delete a file and verify reference-count updates.

Fault-tolerance tests

- Stop one node and download a replicated file.
- Corrupt a chunk and confirm hash verification fails.
- Remove one replica and confirm recovery creates another copy.
- Stop a node during upload and verify the operation reports failure safely.

Performance metrics

```
Storage overhead = physical storage / logical file size
Deduplication ratio = logical data / unique physical data
Availability = successful requests / total requests
Recovery time = recovery completion time - failure detection time
```

Use files of 1 MB, 10 MB, 50 MB, and 100 MB. Record actual measurements rather than estimated values.

12. Risk analysis and mitigation

Risk	Impact	Mitigation
Metadata server failure	High	Document as limitation; add metadata backup
Node failure	Medium	Maintain a second copy and recovery process
Corrupted chunk	High	Verify SHA-256 before storage and download
Duplicate data	Medium	Use content-addressed storage and reference counting
Insufficient disk space	Medium	Monitor capacity and reject placement when necessary
Concurrent uploads	Medium	Use transactions, temporary files, and unique IDs
Path traversal	High	Sanitize file names and use hash-based paths
Schedule delay	High	Complete MVP before optional erasure coding

13. Limitations and future work

Limitations

- SQLite is suitable for a prototype but not a large production cluster.
- Running all nodes on one laptop does not reproduce independent hardware failures.
- A single metadata server is a potential single point of failure.
- Fixed-size chunks are simpler but may be less effective for modified files.
- Replication consumes more storage than erasure coding.

Future work

- Replicated metadata server.
- Content-defined chunking such as FastCDC. FastCDC research focuses on improving chunking speed while preserving deduplication effectiveness. [3]
- Reed-Solomon erasure coding.
- End-to-end encryption and secure key management.
- TLS communication between services.
- Distributed database for metadata.
- Automatic load balancing and rebalancing.
- Cloud deployment across independent machines.

14. Team responsibilities

Role	Responsibility
Project coordinator	Requirements, schedule, integration, and final demonstration
Metadata developer	Database, metadata API, and node-location management
Storage developer	Storage-node API, chunk files, and health endpoint
Reliability developer	Deduplication, replication, failure detection, and recovery
Documentation and testing lead	Test cases, measurements, diagrams, report, and presentation

Use Git branches for separate modules and merge tested work into the main branch.

15. Presentation structure

Slide 1: Title

Project title, team members, course, and supervisor.

Slide 2: Problem

Single-server storage has failure, capacity, corruption, and duplication problems.

Slide 3: Objectives

Show the main project objectives.

Slide 4: Proposed solution

Explain chunking, metadata, storage nodes, and reconstruction.

Slide 5: Architecture

Present the client, metadata server, and three storage nodes.

Slide 6: Upload and download workflow

Use arrows to show the two workflows.

Slide 7: ER diagram

Explain the major entities and relationships.

Slide 8: Storage optimization

Compare one copy, two replicas, deduplication, and optional $(4, 2)$ erasure coding.

Slide 9: Fault tolerance

Demonstrate a failed node and recovery from the second copy.

Slide 10: Feasibility

Present technical, economic, operational, and schedule feasibility.

Slide 11: Testing and results

Show actual upload time, download time, storage overhead, and recovery time.

Slide 12: Limitations and future work

Mention metadata-server redundancy, encryption, erasure coding, and cloud deployment.

Slide 13: Conclusion

Summarize the contribution and answer questions.

16. Conclusion

The proposed Distributed File Storage System is a feasible and academically appropriate semester project for the Distributed Systems course. It demonstrates data partitioning through chunking, distributed placement across nodes, metadata management, cryptographic integrity verification, deduplication, replication, failure detection, and recovery.

The recommended project boundary is a working three-node prototype with fixed-size chunks, SHA-256, deduplication, replication factor two, and automatic recovery. This provides a complete and demonstrable system. Erasure coding and content-defined chunking should be presented as advanced extensions or implemented only after the MVP is stable.

References

- [1] Apache Software Foundation, “HDFS Architecture,” Apache Hadoop Documentation. <https://hadoop.apache.org/docs/r3.4.1/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- [2] Ceph Documentation, “Erasure Code,” <https://docs.ceph.com/en/latest/rados/operations/erasure-code/>
- [3] Y. Xia et al., “FastCDC: A Fast and Efficient Content-Defined Chunking Approach for Data Deduplication,” USENIX ATC 2016. <https://www.usenix.org/conference/atc16/technical-sessions/presentation/xia>